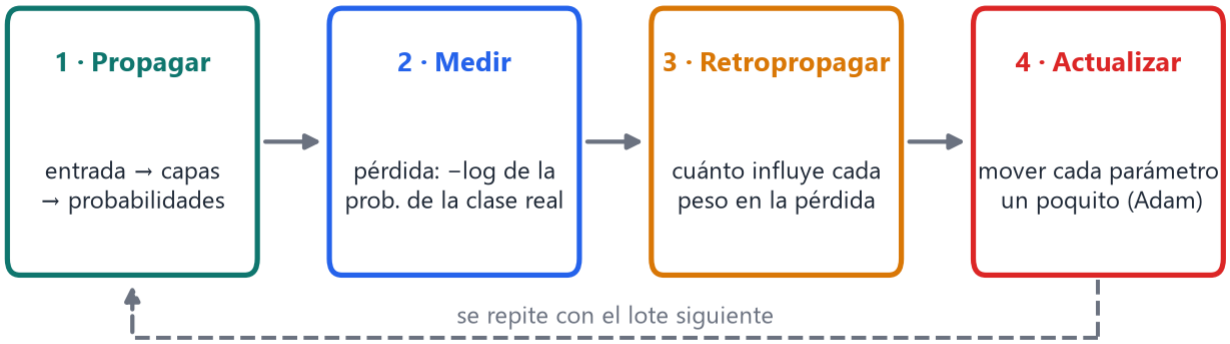
Clase 45 · Redes neuronales

Profesor Eloy Alvarado · primera mirada al interior de una red: neurona, capas, aprendizaje por gradiente y evaluación, con los dígitos de la Clase 44

La gran idea de la clase

Una red neuronal es una cadena de operaciones muy simples. Cada neurona hace una suma ponderada de sus entradas, le suma un sesgo y aplica una función de activación; las salidas de una capa son las entradas de la siguiente; y al final **softmax** convierte los números en 10 probabilidades (una por dígito). Para **aprender**, la red mide su error con la pérdida logarítmica, calcula cuánto influye cada peso en ese error (**retropropagación**) y mueve cada peso un poco en la dirección que lo reduce (**descenso de gradiente**, aquí con **Adam**), lote tras lote y época tras época. El número de capas y neuronas, el tamaño del paso, el lote, alpha y las épocas son **hiperparámetros**: se fijan antes de entrenar y se eligen con la **validación**, nunca con la prueba.

El ciclo de entrenamiento en una imagen



Tras cada época (una pasada por todas las imágenes de entrenamiento) se mira la validación.

Lo que necesitas ANTES

- **Regresión logística y probabilidades** (Clases 20–21 y 31): una probabilidad por clase
- **Log loss y exactitud** (Clases 38 y 44): −log de la probabilidad de la clase real
- **Validación y prueba** (Clases 30 y 38): elegir con una, evaluar al final con otra
- **Idea de derivada** (pendiente de una curva) y **multiplicar matrices**: se repasan aquí con números pequeños

Ruta de esta guía

- Neurona → activaciones → capa y matrices → arquitectura 64–32–10 y parámetros
- Salida softmax → pérdida → objetivo con penalización → descenso de gradiente → retropropagación
- Lotes, épocas, Adam e inicialización → curvas, ancho y alpha
- Evaluación: log loss vs errores, matriz de confusión, errores seguros y límites

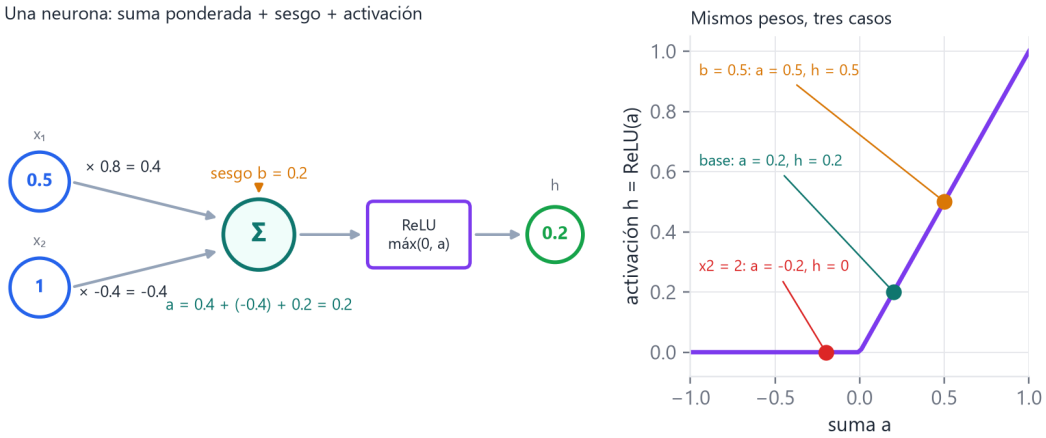
Cómo leer los números de este material

Las **diapositivas** muestran la corrida del profesor. Aquí se ejecutó **su mismo programa** (`redes_neuronales.py`, scikit-learn 1.9.1, misma semilla) y **las 12 cifras comparadas coinciden exactamente** con las diapositivas (tabla en la última página). Lo que se rotula «nuestra reproducción» (otros anchos, barrido de alpha, Adam sobre el juguete) es cálculo propio, no de la clase.

1 La neurona: una suma ponderada con un interruptor

Una neurona recibe números, los multiplica por pesos, suma un sesgo y pasa el resultado por una función de activación. Eso es todo: $a = w_1x_1 + \dots + w_px_p + b$, y la salida es $h = \varphi(a)$.

1. **Entradas x.** Los números que llegan: en la clase, las 64 intensidades de una imagen (entre 0 y 1) o las salidas de la capa anterior.
2. **Pesos w.** Cuánta importancia da la neurona a cada entrada. Un peso positivo hace subir la suma; uno negativo, bajarla. Son **parámetros**: se aprenden.
3. **Sesgo b.** Un número que desplaza toda la suma. Permite que la neurona responda aunque todas las entradas sean 0. También es un parámetro.
4. **Activación $h = \varphi(a)$.** El resultado de aplicar una función a la suma. Con ReLU, $\varphi(a) = \max(0, a)$: si a es negativo, la neurona entrega 0.



Diapositivas 5 y 6: una neurona con x = (0.5, 1), pesos (0.8, −0.4) y sesgo 0.2; a la derecha, lo que cambia si cambia x_2 o el sesgo.

En palabras simples

Piensa en un **jurado de dos personas**: cada una da su opinión (x), tú le crees más a una que a otra (los pesos), partes con una predisposición (el sesgo) y decides «sí» solo si el balance es positivo (ReLU). Los pesos no cambian entre una imagen y otra; lo que cambia es la opinión que entra, y por eso cambia la respuesta.

$$a = w_1x_1 + w_2x_2 + b$$

$$h = \text{ReLU}(a) = \max(0, a)$$

Con los números de la clase

- Caso base: $a = 0.8(0.5) - 0.4(1) + 0.2 = 0.2 \rightarrow h = 0.2$
- Con $x_2 = 2$: $a = 0.4 - 0.8 + 0.2 = -0.2 \rightarrow$ ReLU lo lleva a $h = 0$
- Con $b = 0.5$ y $x_2 = 1$: $a = 0.4 - 0.4 + 0.5 = 0.5 \rightarrow h = 0.5$
- ReLU(−2) = 0 y ReLU(3) = 3 (diapositiva 5)

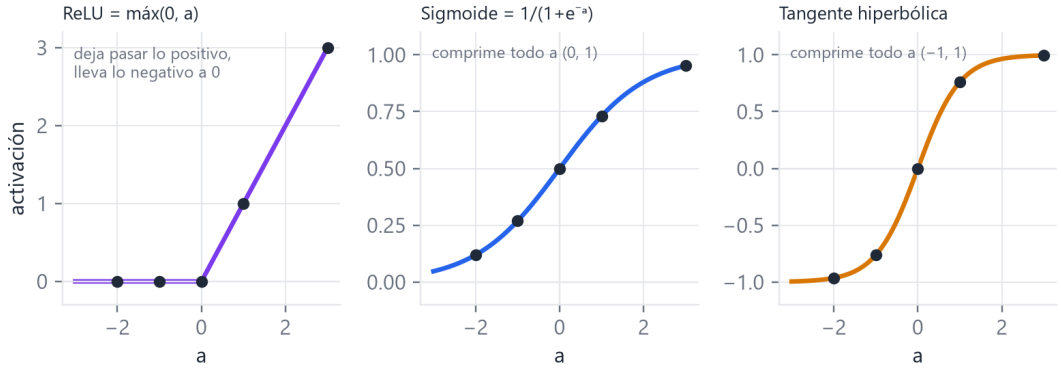
Cuidado. «Activación» no significa «distinta de cero»: una activación de 0 es un resultado válido. Y las activaciones **no son probabilidades** ni representan un dígito en particular.

2 Funciones de activación y por qué hace falta la no linealidad

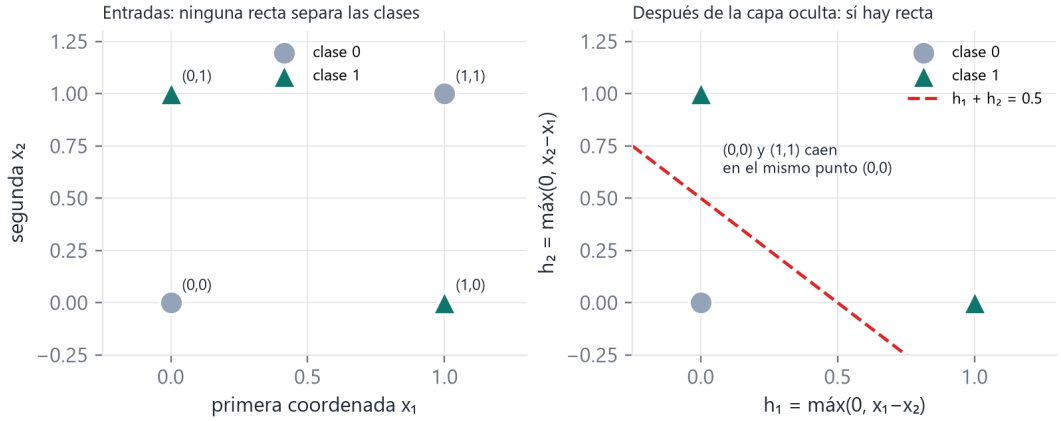
La suma ponderada es una función lineal; sin una activación no lineal entre capas, apilar capas no aporta nada. Dos capas lineales seguidas equivalen a una sola capa lineal.

1. **ReLU.** Deja pasar lo positivo y lleva lo negativo a 0. Puede superar 1. Es la habitual en capas ocultas.
2. **Sigmoide.** Comprime cualquier valor al intervalo (0, 1): sirve para salidas binarias que se leen como probabilidad ($\sigma(0) = 0.5$).
3. **Tangente hiperbólica.** Comprime al intervalo (−1, 1); $\tanh(0) = 0$.

4 **Transformar el espacio.** Con una capa oculta no lineal, puntos que ninguna recta separaba pasan a poder separarse (ejemplo de la diapositiva 11).



Diapositivas 5 y 7: las tres funciones; los puntos negros son a = −2, −1, 0, 1 y 3.



Diapositiva 11: clase 1 si las dos coordenadas son distintas. Con $h_1 = \max(0, x_1 - x_2)$ y $h_2 = \max(0, x_2 - x_1)$, una recta ($h_1 + h_2 = 0.5$) separa las clases. Los pesos se fijaron a mano.

En palabras simples

Imagina que tienes **reglas transparentes**: poner varias reglas rectas una encima de otra sigue dando una línea recta. La activación es como **doblar** la regla en un punto: con dobleces se puede formar curvas. Y en el ejemplo, la capa oculta «pliega» el cuadrado de modo que dos esquinas opuestas quedan juntas y las otras dos separadas.

$$s = (xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$$

→ una sola capa lineal con W* y b*

Con los números de la clase

- Sin activación, con x = (3, −1): dos capas dan 13.5 y la capa única equivalente también da 13.5 (ejemplo propio)
- Tabla de la diapositiva 11: (0,0) → (0,0); (1,1) → (0,0); (1,0) → (1,0); (0,1) → (0,1); $h_1 + h_2$ vale 0, 0, 1, 1
- Cuándo usar cuál: ReLU en capas intermedias; sigmoide (o softmax) en la salida de clasificación; salida lineal si se predice un número

Cuidado. La función de activación de la **salida** depende de la tarea (diapositiva 16): ReLU no sirve para producir probabilidades porque no está acotada.

3 Una capa: muchas neuronas a la vez, con matrices

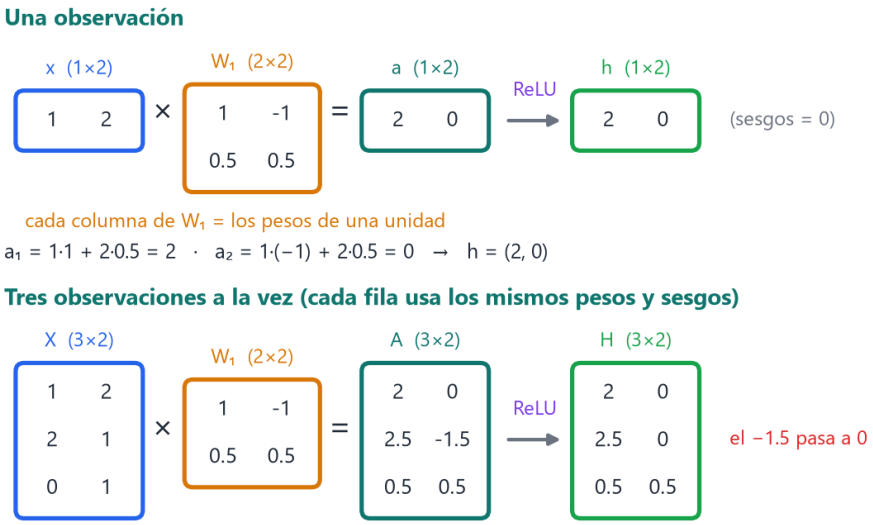
Una capa reúne varias neuronas que reciben la misma entrada pero tienen pesos y sesgos propios. Calcularlas todas es un producto de matrices: X(n×p) · W(p×q) = A(n×q), más el sesgo y la activación.

1 **Cada columna de W es una neurona.** Con 2 entradas y 2 unidades, W es 2×2: la primera columna tiene los pesos de la unidad 1; la segunda, los de la unidad 2.

2 **Cada fila de X es una observación.** Se pueden procesar 3 imágenes (o 360) de una vez con una sola multiplicación: cada fila usa los mismos pesos y sesgos.

3 **Dimensiones.** n observaciones × p entradas, por p × q unidades, da n × q activaciones. El sesgo (q números) se suma a todas las filas.

4 **Con imágenes.** En la clase, X es 360 × 64 (imágenes de validación), W_1 es 64 × 32 y H es 360 × 32.



Diapositivas 8, 12 y 14: una observación x = (1, 2) y tres observaciones a la vez; sesgos en cero. En la fila 2, $a_2 = -1.5$ pasa a 0 por ReLU.

En palabras simples

Es como una **planilla de cálculo**: cada columna de W es una «receta» con sus propios ingredientes (pesos) y cada fila de X es un «pedido». La multiplicación aplica todas las recetas a todos los pedidos de una sola vez.

$a_1 = 1 \cdot 1 + 2 \cdot 0.5 = 2 \rightarrow h_1 = 2$

$a_2 = 1 \cdot (-1) + 2 \cdot 0.5 = 0 \rightarrow h_2 = 0$

Con los números de la clase

- x = (1, 2) con $W_1 = [[1, -1], [0.5, 0.5]]$ y $b_1 = (0, 0)$ da h = (2, 0)
- Fila 2 de X, (2, 1): a = (2·1 + 1·0.5, 2·(−1) + 1·0.5) = (2.5, −1.5) → h = (2.5, 0)
- Fila 3, (0, 1): a = (0.5, 0.5) → h = (0.5, 0.5)

Cuidado. Hay que usar siempre la misma convención: aquí cada observación es una **fila** y se multiplica x·W (no W·x).

4 Leer una arquitectura y contar sus parámetros: 64–32–10

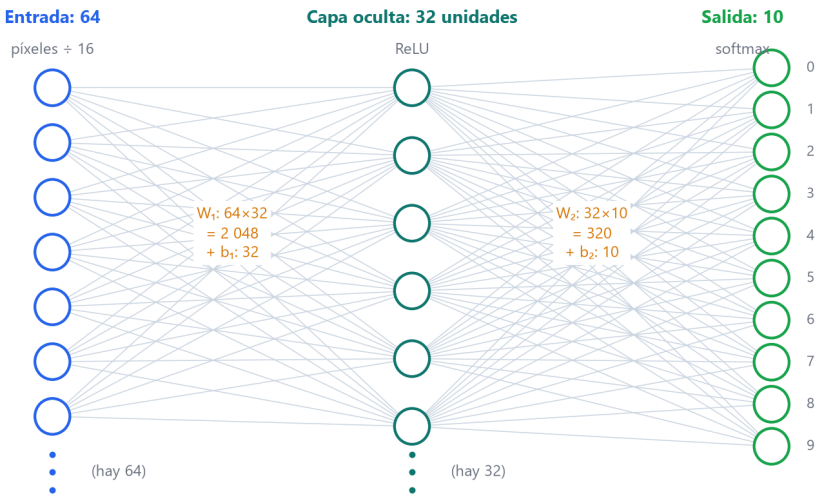
Se lee de izquierda a derecha: 64 entradas, 32 unidades ocultas, 10 salidas. Al ajustar, la red 64–32–10 tiene 2 410 parámetros (pesos y sesgos); la 64–16–10, 1 210.

1 **Capa oculta y ancho.** Las capas entre la entrada y la salida. El número de unidades de una capa es su **ancho**; el número de capas, su **profundidad**.

2 **Hiperparámetro.** Se fija antes de ajustar con los datos (32, 16, número de capas, activación…). No hay valor óptimo conocido de antemano: hay que probar.

3 **Parámetro.** Se aprende al entrenar: los pesos W y los sesgos b. No se cuentan las activaciones ni los píxeles.

4 **Red densa.** Cada unidad recibe las 64 entradas. Otras arquitecturas (convolucionales) conectan solo partes de la imagen.



Parámetros: 2 048 + 32 + 320 + 10 = 2 410 (con 16 unidades: 64·16+16 + 16·10+10 = 1 210)

Diapositivas 9 y 13: se dibujan 7 de las 64 entradas y 6 de las 32 unidades ocultas; las 10 salidas son los dígitos 0 a 9.

En palabras simples

Como en una **cadena de montaje**: 64 piezas entran, 32 estaciones intermedias las combinan cada una a su manera y 10 estaciones finales entregan un puntaje por dígito. Cada flecha del dibujo es un peso; cada estación tiene además su sesgo. Más estaciones = más capacidad, pero también más cosas que ajustar y más tiempo.

Ancho 32: 32(64 + 1) + 10(32 + 1) = 2080 + 330 = 2 410

Ancho 16: 16(64 + 1) + 10(16 + 1) = 1040 + 170 = 1 210

Con los números de la clase

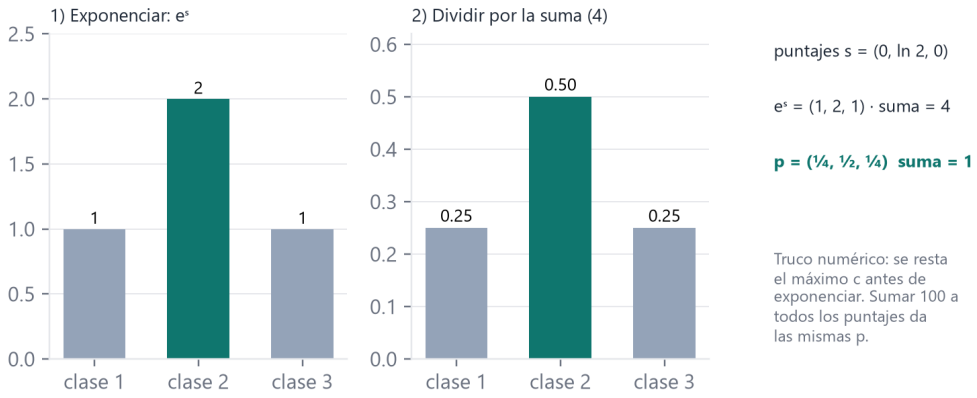
- $W_1 = 64 \times 32 = 2\,048 \cdot b_1 = 32 \cdot W_2 = 32 \times 10 = 320 \cdot b_2 = 10$ (diapositiva 13)
- La regresión logística sobre los mismos píxeles tiene 64·10 + 10 = 650 parámetros (nuestro cálculo)
- Agregar una segunda capa oculta de 32 a la 64–32–10 da 3 466 parámetros, no el doble (ejemplo propio)
- Los modelos actuales tienen millones o miles de millones de parámetros ya ajustados (comentado en clase)

Cuidado. Contar parámetros es contar columnas: cada unidad tiene (entradas + 1) parámetros, donde el +1 es el sesgo.

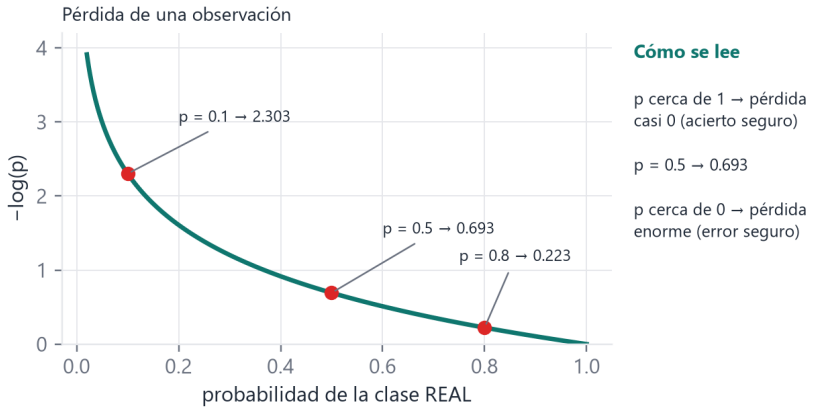
5 La salida: softmax y la pérdida de una observación

Para clases excluyentes, la última capa usa softmax: convierte 10 puntajes cualquiera en 10 probabilidades que suman 1. La pérdida de una imagen es −log de la probabilidad que se dio a la clase real.

1. **Tarea y salida.** Regresión: un número (pérdida: error cuadrático). Binaria: una sigmoide (entropía cruzada binaria). Clases excluyentes: softmax (entropía cruzada multiclase). Salida y pérdida deben corresponder a la tarea.
2. **Softmax.** Se exponencia cada puntaje y se divide por la suma de los exponenciales. Todos quedan entre 0 y 1 y suman 1.
3. **Pérdida −log p.** Usa la probabilidad de la clase **real**, aunque otra clase tenga mayor probabilidad. Castiga fuerte los errores seguros.
4. **Truco numérico.** Restar el máximo c a los puntajes antes de exponenciar no cambia las probabilidades y evita números enormes.



Diapositiva 17: puntajes s = (0, ln 2, 0) → exponenciales (1, 2, 1), suma 4 → probabilidades (0.25, 0.50, 0.25).



Diapositiva 18: pérdida = −log p para p = 0.8, 0.5 y 0.1.

En palabras simples

Softmax es como **repartir 100 puntos de confianza** entre las 10 opciones según cuánto destaca cada puntaje (los exponenciales exageran las diferencias). La pérdida es la **multa** por haber apostado poco a la respuesta correcta: con 80 % de apuesta la multa es pequeña; con 10 %, grande.

$p_k = e^{s_k} \div \Sigma_j e^{s_j}$

pérdida = −log p(clase real)

Con los números de la clase

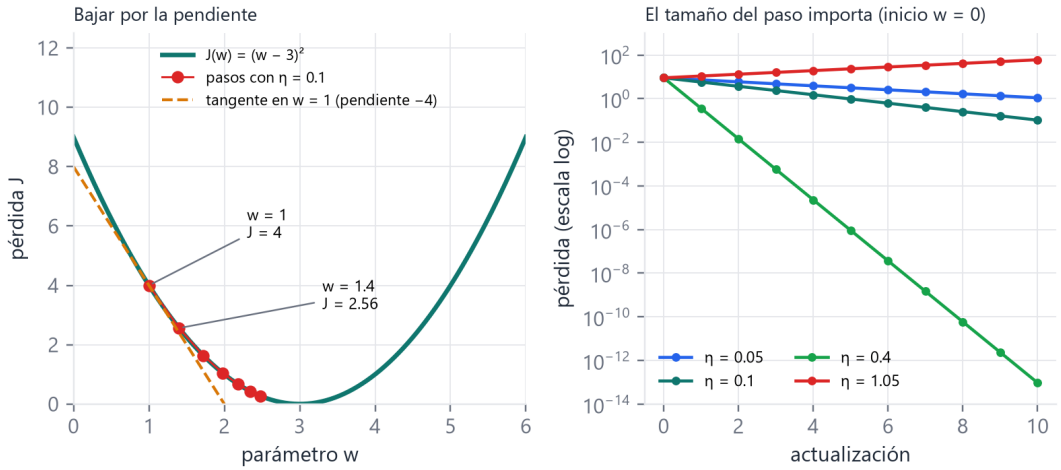
- s = (0, ln 2, 0) → p = (¼, ½, ¼): 0.25, 0.50, 0.25
- −log 0.8 = 0.223 · −log 0.5 = 0.693 · −log 0.1 = 2.303 (diapositiva 18)
- Sumar 100 a todos los puntajes da las mismas probabilidades (diapositiva 17)
- Ejemplo binario de la diapositiva 15: s = 0.5 → σ(0.5) = 0.6225 (con 10 clases no basta una sigmoide)

Cuidado. Las salidas de softmax se **interpretan** como probabilidades; no garantizan estar bien calibradas (la clase pasada ya mostró errores confiados).

6 Qué es «aprender»: minimizar J con el descenso de gradiente

Aprender es buscar los pesos y sesgos θ que minimizan el objetivo J(θ) = pérdida media + penalización por pesos grandes. El gradiente indica hacia dónde sube J; se avanza en sentido contrario.

1. **Objetivo J.** Primer término: el promedio de las pérdidas de las imágenes (ajuste a los datos). Segundo: λ/2 por la suma de los pesos al cuadrado (preferencia por pesos pequeños). En scikit-learn el control es **alpha**.
2. **Gradiente.** La derivada en varias dimensiones: una pendiente por cada parámetro. Si la pendiente es negativa, aumentar el parámetro baja J.
3. **Regla de actualización.** $w^+ = w - \eta \cdot J'(w)$. η (eta) es la **tasa de aprendizaje**, el tamaño del paso.
4. **Tamaño del paso.** Con η muy pequeño se avanza lento; con η muy grande se pasa de largo y J puede empezar a crecer.



Diapositivas 20 y 27: $J(w) = (w-3)^2$. Izquierda: pasos de $\eta = 0.1$ desde $w = 1$. Derecha: pérdida tras cada actualización con cuatro tasas (inicio $w = 0$).

En palabras simples

Es como **bajar un cerro con niebla**: no ves el valle, pero sientes con los pies hacia dónde baja el suelo y das un paso en esa dirección. Si los pasos son diminutos, tardas horas; si son enormes, saltas al otro lado del valle y terminas más arriba.

$$J'(w) = 2(w - 3)$$

$$w^{+} = w - \eta \cdot J'(w)$$

Con los números de la clase

- $w = 1$: $J = (1-3)^2 = 4$; $J'(1) = 2(1-3) = -4$; $w^{+} = 1 - 0.1(-4) = 1.4$; $J(1.4) = 2.56$
- Restar una derivada negativa aumenta el parámetro, y aquí eso reduce la pérdida
- Pérdida tras 10 actualizaciones desde $w = 0$: $\eta = 0.05 \to 1.09$; $\eta = 0.1 \to 0.104$; $\eta = 0.4 \to 9.4e\text{-}14$; $\eta = 1.05 \to 60.5$ (cálculo propio: factor $(1-2\eta)^2$ por paso)
- $(-w)^2 = w^2$ y $(2w)^2 = 4w^2$: la penalización mira la magnitud y crece al cuadrado

Cuidado. $J(w) = (w - 3)^2$ es un **juguete**: la pérdida real de la red depende de miles de parámetros. Con $\eta = 1.05$ la pérdida crece en vez de bajar.

7 Retropropagación: repartir la «culpa» del error hacia atrás

La retropropagación aplica la regla de la cadena para calcular cuánto cambia la pérdida si se mueve cada parámetro. Primero se calcula hacia adelante (predicción y pérdida); luego se vuelve hacia atrás obteniendo una sensibilidad por parámetro.

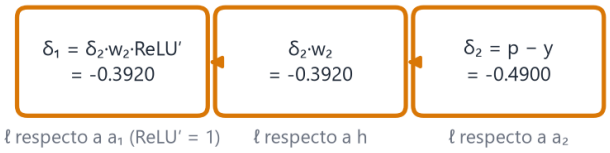
1. **Hacia adelante.** Con $x = 0.5$, $y = 1$ y $\theta = (w_1, b_1, w_2, b_2) = (0.4, 0.1, 0.8, -0.2)$: $a_1 = 0.3$, $h = 0.3$, $a_2 = 0.04$, $p = \sigma(0.04) \approx 0.51$, $\ell = -\log p \approx 0.6733$.
2. **Error en la salida.** Con sigmoide y entropía cruzada binaria, $\delta_2 = \partial \ell / \partial a_2 = p - y$. Aquí $\delta_2 = 0.51 - 1 \approx -0.49$.
3. **Hacia atrás.** $\delta_1 = \delta_2 \cdot w_2 \cdot \text{ReLU}'(a_1) = (-0.49)(0.8)(1) \approx -0.392$. Luego: $\partial \ell / \partial w_2 = \delta_2 h$, $\partial \ell / \partial b_2 = \delta_2$, $\partial \ell / \partial w_1 = \delta_1 x$, $\partial \ell / \partial b_1 = \delta_1$.
4. **Qué hace el optimizador.** Backpropagation solo calcula los gradientes. El **optimizador** decide cómo cambiar los parámetros con ellos.

Hacia adelante (propagación): se calculan los valores



pérdida ℓ = −log p = 0.6733 (y = 1)

Hacia atrás (retropropagación): se calcula cuánto influye cada cosa en ℓ



Sensibilidades de los cuatro parámetros (θ = w_1, b_1, w_2, b_2):

$\partial \ell/\partial w_1 = \delta_1 \cdot x = -0.1960$ · $\partial \ell/\partial b_1 = \delta_1 = -0.3920$ · $\partial \ell/\partial w_2 = \delta_2 \cdot h = -0.1470$ · $\partial \ell/\partial b_2 = \delta_2 = -0.4900$

Diapositivas 21 a 23: el ejemplo completo, con la propagación (arriba) y la retropropagación (abajo).

En palabras simples

Es una **cadena de responsabilidades**: al final alguien dice «salió mal por 0.49». Esa persona le avisa a la anterior cuánto pesó su parte (multiplicando por su peso w_2), ella avisa a la anterior, y así hasta los primeros pesos. Cada uno aprende cuánto le tocó de la culpa y corrige solo su parte.

$\delta_2 = p - y$

$\delta_1 = \delta_2 \cdot w_2 \cdot \text{ReLU}'(a_1)$

$\partial \ell/\partial w = \delta \cdot$ (entrada de ese peso)

Con los números de la clase

- □ℓ = (-0.1960, -0.3920, -0.1470, -0.4900)
- Paso con η = 0.1: θ^+ = (0.4196, 0.1392, 0.8147, -0.1510)
- Nueva predicción: $a_1 \approx 0.3490$, $a_2 \approx 0.1333$, $p \approx 0.533283$; pérdida 0.673347 → 0.628702 (baja)
- Comprobación numérica (diferencias finitas): el gradiente coincide con diferencia máxima 8e-11

Cuidado. Un paso que baja la pérdida de este ejemplo **no demuestra** que la red generalice. Las cifras de la diapositiva están redondeadas.

8 Entrenar en la práctica: lotes, épocas, Adam e inicialización

No se actualiza con una sola imagen ni con todas a la vez: se reparten en lotes. Un lote produce una actualización; recorrer todo el entrenamiento es una época. El optimizador Adam decide el tamaño de cada paso.

1. **Lote, actualización y época.** Lote: un grupo de imágenes. Actualización: un cambio de parámetros con un lote. Época: una pasada por todas las imágenes de entrenamiento.
2. **Adam.** Un optimizador que adapta el paso de cada parámetro: promedia los gradientes recientes (inercia) y los divide por su tamaño típico. La clase solo lo nombró; aquí se ilustra con el juguete.
3. **Inicialización.** Hay que partir de valores iniciales. Si todas las unidades parten con los mismos pesos, calculan lo mismo y se mantienen iguales: conviene que difieran.
4. **Cuándo parar.** Cuando la pérdida de validación deja de mejorar o se agota el presupuesto de épocas (120 en la clase).

Una época = una pasada por las 1 077 imágenes de entrenamiento



16 lotes de 64 imágenes + 1 lote final de 53 = 17 actualizaciones por época

17 actualizaciones × 120 épocas = 2 040 actualizaciones en total

Diapositiva 26: 1 077 = 16·64 + 53; 17 actualizaciones por época; 120 épocas × 17 = 2 040 actualizaciones.

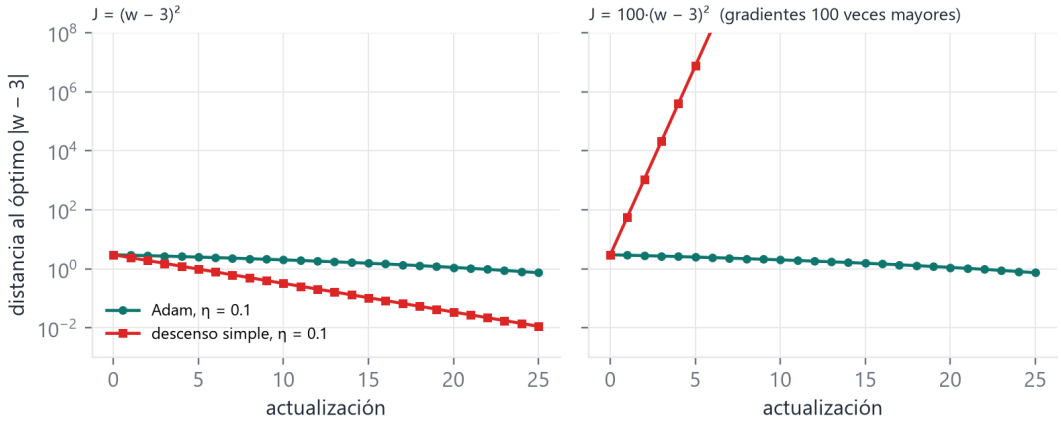


Ilustración propia (no está en las diapositivas): Adam con η = 0.1 avanza ≈ 0.1 por actualización aunque el gradiente sea 100 veces mayor; el descenso simple se desborda.

En palabras simples

Los lotes son como **corregir un examen por montones** de 64 hojas y ajustar tu criterio después de cada montón, en vez de esperar a corregir las 1 077. Adam es un **conductor con inercia**: recuerda hacia dónde venía y frena o acelera según qué tan fuerte es el terreno de cada parámetro.

lotes por época = ⌈1 077 ÷ 64⌉ = 17

Adam: paso ≈ η · (gradiente medio) ÷ (tamaño típico del gradiente)

Con los números de la clase

- 1 077 = 16(64) + 53: el último lote usa las 53 imágenes restantes
- 17 lotes por época × 120 épocas = 2 040 actualizaciones por candidata
- Adam desde w = 0 en $J = (w-3)^2$: primer paso 0.100; con $J = 100(w-3)^2$ el gradiente inicial es 600 en vez de 6 y el primer paso sigue siendo 0.100 (cálculo propio)
- Tres unidades con pesos idénticos 0.5 y x = (1, 2) dan h = [1.5, 1.5, 1.5]: las tres activaciones son iguales (diapositiva 28)

Cuidado. Receta del profesor: Adam, tasa 0.002, lote 64, 120 épocas. **Alpha no es la tasa de aprendizaje** (en la clase se mencionó «alpha … ese eta»): alpha controla la penalización; la tasa es `learning_rate_init`.

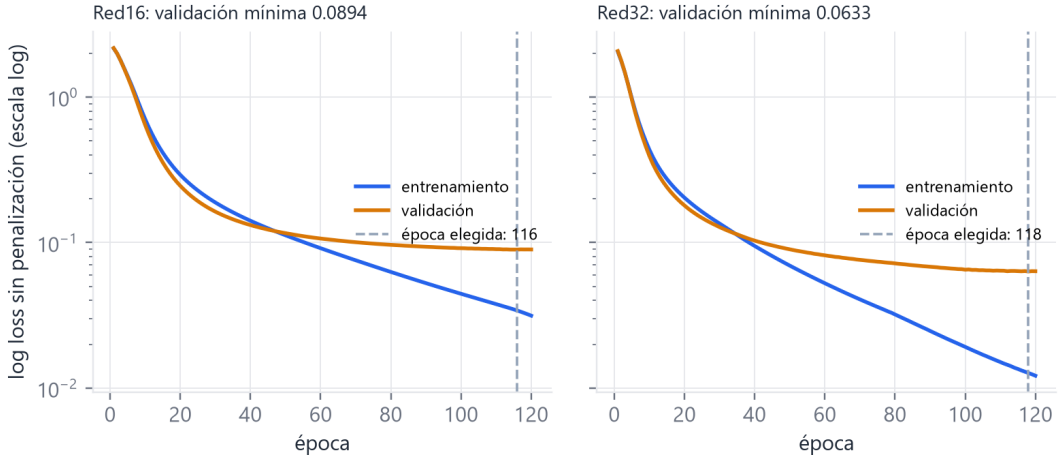
9 Leer las curvas y elegir el ancho con la validación

Cada candidata se entrena 120 épocas y se conserva la época con menor log loss de validación. La elegida fue Red32 (log loss de validación 0.0633, época 118) frente a Red16 (0.0894, época 116).

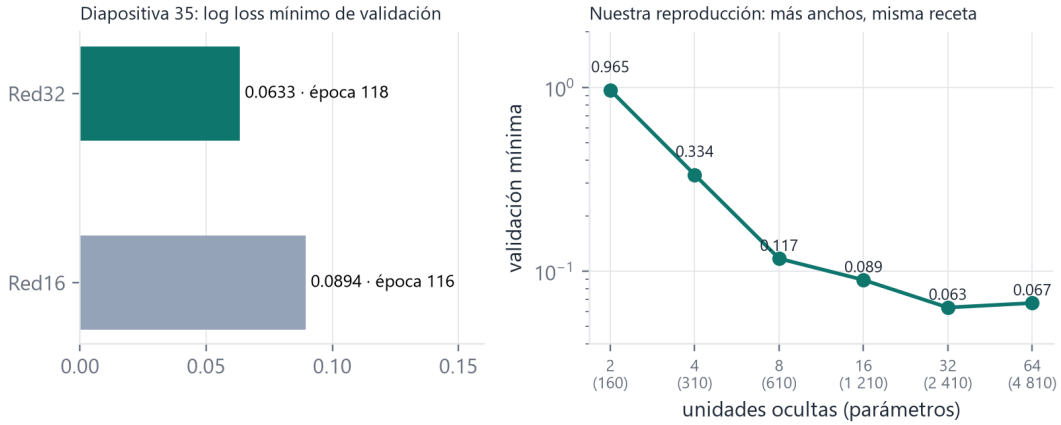
1. **Dos curvas.** Entrenamiento y validación, ambas como log loss sin penalización. Con el tiempo, la de entrenamiento sigue bajando; la de validación se aplana.
2. **Qué indica una brecha.** Si entrenamiento baja pero validación no, la red empieza a memorizar los datos (sobreajuste).

3 **Elegir por validación.** Se comparan los mínimos de validación de cada candidata. No se mira la prueba para elegir, y no se reajusta con entrenamiento + validación.

4 **Sin afirmar convergencia.** Las épocas elegidas (116 y 118) están cerca del límite de 120: no se afirma que la red haya convergido.



Diapositiva 36: log loss por época (escala logarítmica para ver los detalles). Línea punteada: época elegida.



Izquierda, diapositiva 35. Derecha, nuestra reproducción: misma receta con 2, 4, 8, 16, 32 y 64 unidades ocultas.

En palabras simples

Estudiar para un examen usando **solo el libro de ejercicios** hace que cada vez lo sepas mejor (curva azul), pero lo que importa es cómo te va en **problemas nuevos** (curva naranja). Cuando el libro sigue mejorando y los problemas nuevos no, estás memorizando.

Con los números de la clase

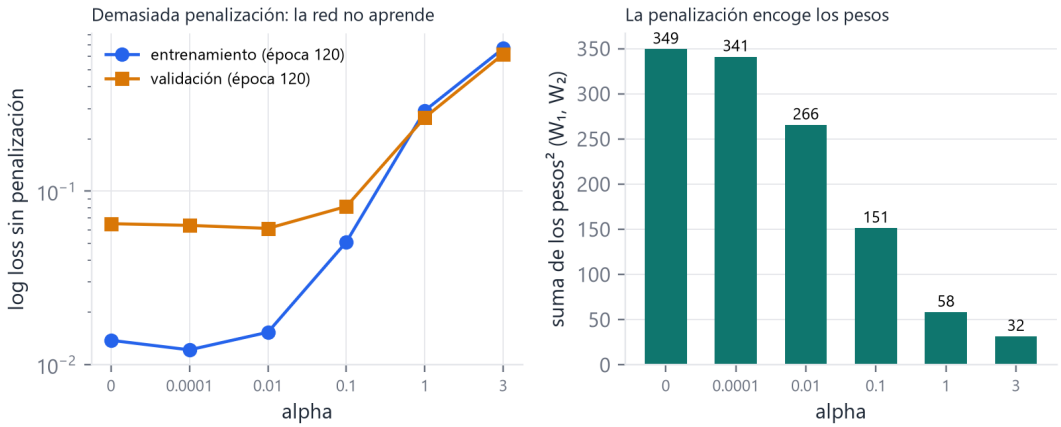
- Red16: entrenamiento 0.0314 y validación 0.0896 en la época 120 (nuestro cálculo con el programa del profesor)
- Red32: entrenamiento 0.0122 y validación 0.0635 en la época 120
- Nuestra reproducción: 2 unidades → 0.965; 4 → 0.334; 8 → 0.117; 16 → 0.089; 32 → 0.063; 64 → 0.067 (el mínimo está en 32)
- Una semilla y una cuadrícula pequeña no establecen estabilidad general (diapositiva 43)

Cuidado. Que 32 gane a 16 en esta corrida no prueba que 32 sea mejor siempre; con otra semilla o con más épocas el resultado podría cambiar.

10 Regularización: el papel de alpha

La penalización de pesos grandes (alpha en scikit-learn) controla cuánto se prefiere una red «suave» frente al ajuste puro a los datos. Poco alpha deja a la red libre; demasiado alpha le impide aprender.

1. **Qué penaliza.** La suma de los pesos al cuadrado (los sesgos no). Duplicar un peso cuadruplica su aporte a la penalización.
2. **Poco alpha.** La red se ajusta casi sin freno: el entrenamiento llega a pérdidas muy bajas.
3. **Mucho alpha.** Los pesos se encogen tanto que la red ya no representa bien los datos: sube la pérdida de entrenamiento y la de validación.
4. **Se elige con validación.** Como el ancho y la tasa: es un hiperparámetro. La receta del profesor usa alpha = 0.0001 (diapositiva 33).



Nuestra reproducción (no está en las diapositivas): red 64–32–10, misma receta, solo cambia alpha. Izquierda: log loss sin penalización tras 120 épocas. Derecha: suma de los pesos al cuadrado.

En palabras simples

Es como el **consejo de una persona prudente** que dice «no exageres»: con poco consejo, la red exagera sus pesos para calzar con cada detalle de los datos; con demasiado, la red ya no se atreve a aprender nada. El punto bueno está en medio.

$J(\theta)$ = promedio de pérdidas + $(\lambda/2)\cdot\Sigma\|W\|^2$

scikit-learn: penalización = $\alpha/(2\cdot\text{lote})\cdot\Sigma W^2$

Con los números de la clase

- alpha = 0: validación 0.0650; entrenamiento 0.0138; suma de W^2 = 349
- alpha = 0.0001 (receta): validación 0.0635; entrenamiento 0.0122; suma de W^2 = 341
- alpha = 0.01: validación 0.0610; entrenamiento 0.0154; suma de W^2 = 266
- alpha = 0.1: validación 0.0817; alpha = 1: 0.2660; alpha = 3: 0.6179 (la red casi no aprende)

Cuidado. En scikit-learn 1.9.1 el término es $\alpha/(2\cdot\text{tamaño del lote})\cdot\Sigma W^2$ (se verificó en el código fuente): el λ de la diapositiva 19 equivale a alpha ÷ lote. La selección usa log loss sin penalización.

11 Evaluar: la red pierde menos, pero la logística se equivoca menos

En las 360 imágenes de prueba la red tiene menor log loss (0.0932 contra 0.1511) y la logística comete menos errores (8 contra 10). Las dos métricas miden cosas distintas.

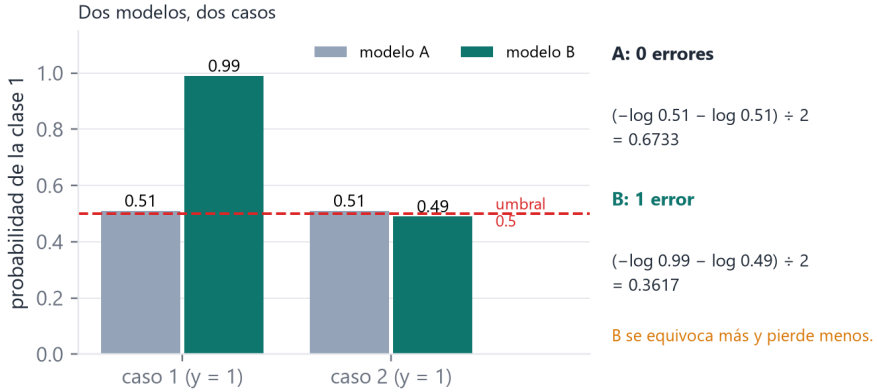
1. **Prueba histórica.** Estas imágenes ya se mostraron en clase: la comparación es **retrospectiva**. Para decidir de verdad, la prueba se mira una sola vez, después de fijar la receta.
2. **Log loss.** Mira las probabilidades: premia la seguridad correcta y castiga la seguridad equivocada.

3 **Exactitud.** Cuenta aciertos con el umbral que decide la clase (0.5 en binario, el máximo en 10 clases). No mira qué tan segura estaba la predicción.

4 **Por qué discrepan.** Un modelo puede equivocarse en más casos pero con probabilidades menos extremas, y pierde menos en promedio.



Diapositiva 37: log loss y errores de la logística y de la red en las mismas 360 imágenes.



Diapositiva 38: en ambos casos y = 1; A acierta los dos con 0.51 (0 errores) y B acierta uno con 0.99 y falla otro con 0.49 (1 error).

En palabras simples

Dos alumnos responden «verdadero o falso» con su **nivel de seguridad**. Uno responde todo con «51 % seguro» y acierta todo; el otro dice «99 % seguro» en uno y «49 %» en otro (falla). El segundo se equivocó una vez, pero su apuesta total estuvo mejor repartida: perdió menos.

$\ell_A = (-\log 0.51 - \log 0.51) \div 2 \approx 0.6733$

$\ell_B = (-\log 0.99 - \log 0.49) \div 2 \approx 0.3617$

Con los números de la clase

- Exactitud: logística 97.78 % (8 errores); red 97.22 % (10 errores)
- Log loss: logística 0.1511; red 0.0932 (diapositiva 37: 0.1511 y 0.0932)
- Validación: log loss logística 0.1434 y red 0.0633
- Ejemplo binario: $\ell_A = 0.6733$ con 0 errores; $\ell_B = 0.3617$ con 1 error

Cuidado. El umbral 0.5 decide la clase; la pérdida evalúa las probabilidades. Elegir según una sola métrica puede llevar a conclusiones distintas.

12 Dónde se equivoca la red y qué no demuestra este experimento

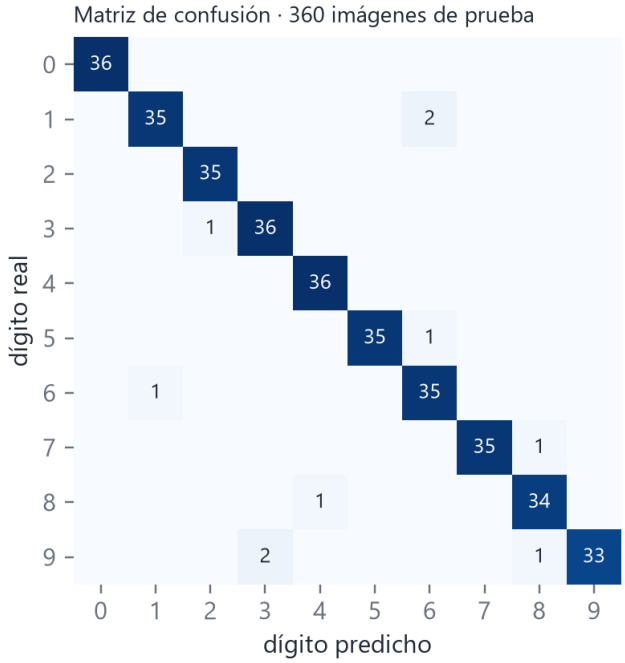
Los 10 errores de la red se reparten entre pocos pares de dígitos y 1 de ellos tiene probabilidad máxima de al menos 0.90. Esos errores seguros son los que más pesan en el log loss.

1 **Matriz de confusión.** Cada fila es el dígito real y cada columna el predicho; la diagonal son los aciertos. Permite ver qué pares se confunden (9 con 8 y 3; 1 con 6).

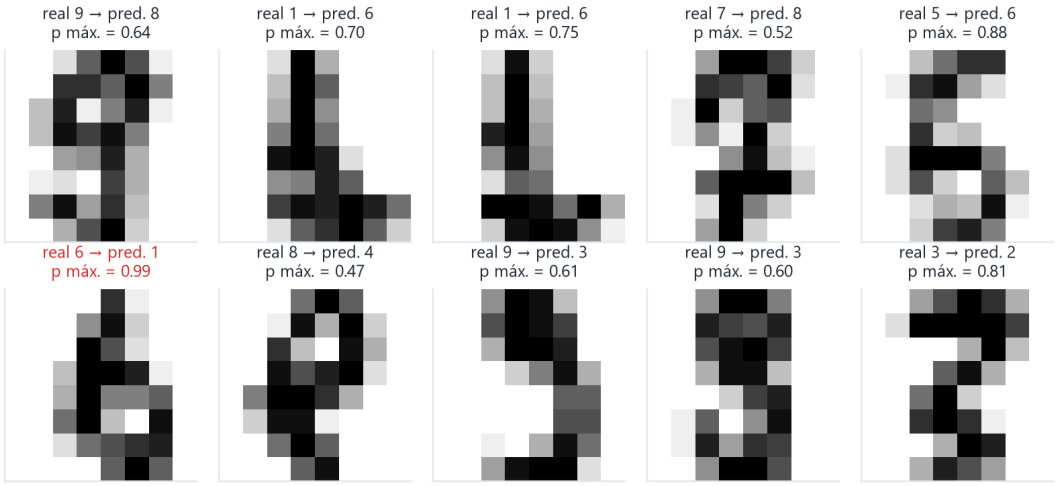
2 **Errores seguros.** El error 6 → 1 tiene p máx. 0.99: la pérdida de esa imagen es 5.75. Los 10 errores suman 26.3 de los 33.5 de pérdida total (78 %).

3 **Alcance.** Una semilla y una cuadrícula chica no dan estabilidad; la prueba reutilizada es retrospectiva; separar por imagen no acredita escritores nuevos; una red densa no es invariante a desplazamientos.

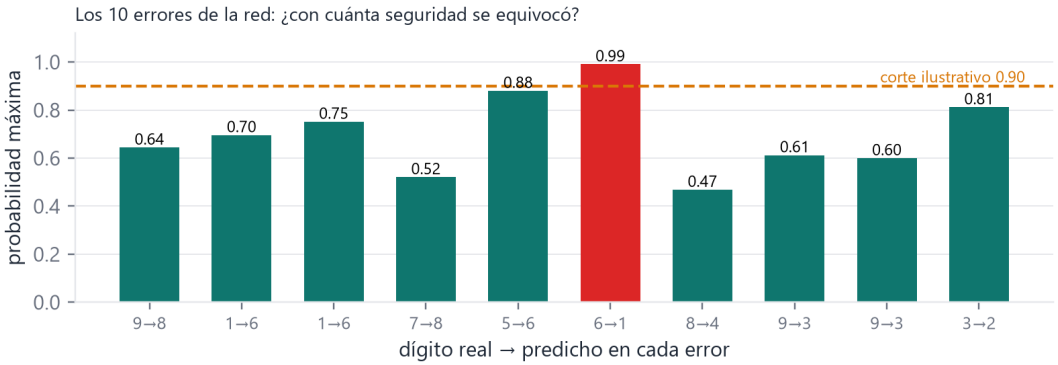
4 **Diagnóstico.** Pérdida no finita → revisar datos, escala, tasa. No mejora el entrenamiento → gradientes, capacidad, tasa, objetivo. Mejora solo el entrenamiento → regularización, época, datos. Falla en otro contexto → distribución y partición.



Diapositiva 39: matriz de confusión de la red en la prueba (360 imágenes).



Diapositivas 40 y 41: los 10 errores de la red en el orden de las filas de prueba (rojo: p máx. ≥ 0.90).



Probabilidad máxima de cada error: solo el 6 → 1 supera el corte ilustrativo de 0.90.

En palabras simples

Una red que **se equivoca con seguridad** es como un guía que te indica el camino equivocado con total convicción: por eso, además de contar errores, hay que mirar las imágenes mal clasificadas y su probabilidad. A veces el error está en la etiqueta del dato, no en el modelo.

Con los números de la clase

- Errores (real → predicho): 9→8, 1→6, 1→6, 7→8, 5→6, 6→1, 8→4, 9→3, 9→3, 3→2
- Primeros seis, p máx.: 0.64, 0.70, 0.75, 0.52, 0.88, 0.99 (diapositiva 40: 0.64, 0.70, 0.75, 0.52, 0.88, 0.99)
- Procedimiento a criticar (diapositiva 45): estandarizar antes de separar, elegir la época con mayor exactitud de *prueba*, comunicar solo la pérdida de entrenamiento y concluir que generaliza a escritores nuevos
- Corrección (diapositiva 46): separar por la unidad relevante antes de estimar transformaciones; ajustar con entrenamiento; elegir con validación; evaluar la receta fijada

Cuidado. Una probabilidad alta también puede acompañar un error: no sirve como garantía individual.

Repaso, alertas del material y criterio de dominio

Lo que debes poder explicar sin apuntes antes de la Clase 46 (autoencoders y representaciones)

Alertas del material

Qué conviene saber al leer las diapositivas y la transcripción

- **La transcripción tenía errores de reconocimiento de voz** («redes neuronales» como «reenhorarías» o «regla oral»; «hiperparámetro» como «Iberparámetro»; los decimales sin coma, 0.5 como «05»…). Se corrigieron en la versión limpia; el original se conserva aparte.
- **«Alpha» no es la tasa de aprendizaje.** En la clase se dijo que alpha «hace referencia a ese eta»; en scikit-learn alpha es la penalización de los pesos y la tasa es `learning_rate_init` (0.002).
- **Agregar una capa no «duplica exponencialmente» los parámetros** (comentario oral): añadir una segunda capa de 32 a la 64–32–10 suma 1 056 parámetros (de 2 410 a 3 466).
- **En la clase se dijo «64 unidades»** al contar las unidades de la capa oculta; son 32 (las 64 son las entradas). Y «todas las capas reciben la misma entrada» debe leerse «todas las unidades de una capa».
- **El valor 0.51** de la probabilidad del ejemplo (0.509999) se oyó como «0.599»; los números de las diapositivas son los correctos. El paso a «0.41» es 0.4196 redondeado.
- **Adam solo se nombró.** La explicación de esta guía (inercia + paso adaptativo) es complemento propio, no de la clase.
- **Las cifras de R son otra corrida.** Las diapositivas usan la ejecución de Python; R entrena por separado y puede diferir.

Preguntas de repaso (intenta responder antes de leer)

#	Pregunta	Respuesta corta
1	Una neurona recibe x = (2, −1), pesos (0.5, 1) y sesgo 0.3. ¿Cuál es a y cuál es la activación ReLU?	a = 0.5·2 + 1·(−1) + 0.3 = 0.3; ReLU(0.3) = 0.3.
2	¿Cuántos parámetros tiene una red 64–16–10? ¿Y una 64–32–10?	1 210 y 2 410: 16(64+1) + 10(16+1) y 32(64+1) + 10(32+1).
3	Softmax de (0, ln 2, 0). ¿Qué probabilidades da?	Exponenciales (1, 2, 1), suma 4: (0.25, 0.50, 0.25).
4	Si la clase real recibe p = 0.1, ¿cuánto vale la pérdida? ¿Y con p = 0.8?	−log 0.1 = 2.303; −log 0.8 = 0.223.
5	En $J(w) = (w-3)^2$, con η = 0.1 y w = 1, ¿cuánto vale la actualización?	$J'(1) = -4$; $w^+ = 1 - 0.1(-4) = 1.4$.
6	¿Qué calcula la retropropagación y qué hace el optimizador?	Calcula los gradientes (cuánto influye cada parámetro en la pérdida); el optimizador decide cómo cambiar los parámetros con ellos.
7	1 077 imágenes y lote 64. ¿Cuántas actualizaciones hay por época y cuántas en 120 épocas?	17 (16 lotes de 64 + 1 de 53) y 120 × 17 = 2 040.
8	¿Con qué conjunto se elige entre Red16 y Red32, y qué métrica se usa?	Con validación, por menor log loss sin penalización; la prueba no participa.
9	La red tiene menor log loss pero más errores que la logística. ¿Es contradictorio?	No: el log loss mira las probabilidades y la exactitud solo la clase elegida; pueden diferir.

Criterio de dominio para pasar a la Clase 46

- ☐ Calculo a mano la salida de una neurona y de una capa pequeña, con ReLU
- ☐ Cuento los parámetros de una arquitectura y explico por qué la activación debe ser no lineal
- ☐ Calculo softmax de tres puntajes y la pérdida −log p de la clase real
- ☐ Hago varios pasos de descenso de gradiente en una función de una variable y explico el papel de η
- ☐ Explico qué hace la retropropagación y por qué hay lotes, épocas y un optimizador
- ☐ Elijo entre dos redes con la validación, leo las curvas y explico por qué log loss y exactitud pueden discrepar

Cifras de las diapositivas frente a nuestra reproducción (programa del profesor)

Cifra	Diapositivas	Reproducido	¿Igual?
Log loss de validación, Red16 (época)	0.0894 (116)	0.0894 (116)	sí
Log loss de validación, Red32 (época)	0.0633 (118)	0.0633 (118)	sí
Parámetros Red32 / Red16	2 410 / 1 210	2410 / 1210	sí
Prueba: log loss logística / red	0.1511 / 0.0932	0.1511 / 0.0932	sí
Prueba: errores logística / red (de 360)	8 / 10	8 / 10	sí
Prueba: exactitud logística / red	97.78 % / 97.22 %	97.78 % / 97.22 %	sí
Primeros 6 errores (real→pred, p máx.)	9→8 0.64 · 1→6 0.70 · 1→6 0.75 · 7→8 0.52 · 5→6 0.88 · 6→1 0.99	9→8 0.64 · 1→6 0.70 · 1→6 0.75 · 7→8 0.52 · 5→6 0.88 · 6→1 0.99	sí
Los 10 errores (real→pred)	9→8 1→6 1→6 7→8 5→6 6→1 8→4 9→3 9→3 3→2	9→8 1→6 1→6 7→8 5→6 6→1 8→4 9→3 9→3 3→2	sí
Errores con p máx. ≥ 0.90	1 de 10	1 de 10	sí
Lotes por época / actualizaciones	17 / 2 040	17 / 2040	sí
Ejemplo completo: pérdida antes → después	0.673347 → 0.628702	0.673347 → 0.628702	sí
Ejemplo completo: p antes → después	0.509999 → 0.533283	0.509999 → 0.533283	sí

Nota: la receta de esta clase evaluada en la prueba da 350 aciertos de 360 y log loss 0.0932; las cifras de la red de la Clase 44 (354 de 360 y 0.0657) **no** salen de esta receta: son otra corrida.